ASSIGNMENT – 1

1. Approximation Algorithm and Approximation Ratio

An approximation algorithm is a polynomial-time algorithm that finds a feasible solution close to the optimal solution for an optimization problem. It is useful when finding an exact optimum is computationally expensive, as in many NP-hard problems. Its objective is to provide a good solution efficiently, together with a bound on how far the solution can be from optimal.

Let P be an optimization problem, A an approximation algorithm, and I an instance of P. Let A*(I) denote the optimal objective value and A(I) the value returned by A. The approximation ratio measures the quality of the algorithm's solution. An algorithm with a guaranteed ratio k is called a k-approximation algorithm, where $k \geq 1$.

Approximation Ratio for Minimization

In a minimization problem, the goal is to find a feasible solution with the minimum possible cost. A k-approximation algorithm guarantees that its cost is at most k times the optimal cost.

$$\mathbf{A(I) \leq k\, A^*(I) \quad or \quad A(I) / A^*(I) \leq k}$$

Example: Suppose the optimal cost is 50 and the algorithm returns a solution costing 60.

$$\mathbf{Approximation\ ratio = 60 / 50 = 1.2}$$

Therefore, the returned solution costs 1.2 times the optimum. For a 2-approximation algorithm, $A(I) \leq 2A^*(I)$; its cost is never more than twice the optimal cost.

Approximation Ratio for Maximization

In a maximization problem, the goal is to obtain the maximum possible value. A k-approximation algorithm guarantees at least 1/k of the optimal value.

$$\mathbf{A(I) \geq A^*(I) / k \quad or \quad A^*(I) / A(I) \leq k}$$

Example: If the optimal value is 100 and the algorithm returns 80, the ratio is 100/80 = 1.25. A 2-approximation algorithm guarantees $A(I) \geq A^*(I)/2$, so it obtains at least half the optimal value.

2. 2-Approximation Algorithm for the Vertex Cover Problem

Let G = (V, E) be an undirected graph. A vertex cover is a subset C ⊆ V such that every edge has at least one endpoint in C. The Vertex Cover Problem asks for a vertex cover containing the minimum number of vertices. Let C* be an optimal vertex cover.

Algorithm: APPROX-VERTEX-COVER(G)

1. C ← ∅ and E′ ← E.
2. While E′ is not empty:
   a. Choose any edge (u, v) from E′.
   b. Add both endpoints: C ← C ∪ {u, v}.
   c. Remove from E′ every edge incident on u or v.
3. Return C.

The selected edges are pairwise disjoint because all edges incident on either endpoint are removed after each selection. Thus, the selected edges form a matching. The algorithm adds both endpoints of every selected edge to C, so every original edge is covered: an edge either is selected or is removed because it shares an endpoint with a selected edge.

Proof of the 2-Approximation Guarantee

Suppose the algorithm selects m pairwise disjoint edges. It adds two endpoints per selected edge; therefore, |C| = 2m. Since the selected edges are pairwise disjoint, an optimal vertex cover C* must contain at least one distinct endpoint for each of these m edges. Hence, |C*| ≥ m.

$$\mathbf{|C| = 2m \leq 2|C^*|}$$

Therefore, the algorithm returns a vertex cover whose size is at most twice the optimal size. Hence, APPROX-VERTEX-COVER is a 2-approximation algorithm.

Example

Consider a graph with edges (a,b), (b,c), (c,e), (e,f), (e,d), (d,f), and (d,g). If the algorithm selects three pairwise disjoint edges, it adds their six endpoints to the cover. In the example graph, a cover such as {b, d, e} may cover all edges depending on the exact edge set; the algorithm's selected edges and resulting cover depend on the order in which edges are chosen. The guarantee remains |C| ≤ 2|C*|.

3. Integer Linear Programming Formulation of Vertex Cover

Consider an undirected graph G = (V, E). For each vertex $v \in V$, introduce a binary decision variable:

$$\mathbf{x_v = 1 \text{ if } v \text{ is selected in the vertex cover; otherwise } x_v = 0.}$$

The objective is to minimize the number of selected vertices.

Objective Function

$$\textbf{Minimize} \quad \sum_{v \in V} x_v$$

Constraints

For every edge $(u, v) \in E$, at least one endpoint must be selected. Therefore:

$$x_u + x_v \geq 1 \qquad \text{for every } (u, v) \in E$$

Each vertex is either selected or not selected, so the variables are binary:

$$x_v \in \{0, 1\} \qquad \text{for every } v \in V$$

Complete ILP Model

$$\textbf{Minimize} \quad \sum_{v \in V} x_v$$

$$\textbf{Subject to} \quad x_u + x_v \geq 1, \ \text{for every } (u,v) \in E$$

$$x_v \in \{0,1\}, \ \text{for every } v \in V$$

The objective minimizes the size of the cover, the edge constraints ensure that every edge is covered, and the binary constraints ensure that each vertex is either included or excluded. An optimal solution to this ILP corresponds to a minimum vertex cover.

4. LP-Rounding Algorithm for Vertex Cover

The LP-rounding method relaxes the binary constraints of the ILP and then converts the fractional solution into an integral vertex cover.

Step 1: Relax the ILP

Replace $x_v \in \{0,1\}$ with $0 \le x_v \le 1$. The relaxed linear program is:

$$\textbf{Minimize} \quad \Sigma_{v \in V} x_v$$

$$\textbf{Subject to} \quad x_u + x_v \ge 1, \ \text{for every } (u,v) \in E$$

$$0 \le x_v \le 1, \ \text{for every } v \in V$$

Solve this LP to obtain fractional values for the vertices.

Step 2: Round the Values

For each vertex v:

If $x_v \ge 1/2$, select v; otherwise, do not select v.

Let C be the set of selected vertices.

Why Does Rounding Produce a Vertex Cover?

For every edge (u, v), the LP constraint gives $x_u + x_v \ge 1$. Therefore, at least one endpoint must have a value of at least 1/2. That endpoint is selected during rounding. Thus, every edge has at least one selected endpoint, and C is a valid vertex cover.

Approximation Guarantee

Let x* be an optimal solution of the LP relaxation, with objective value LP*. Every selected vertex has $x_v \ge 1/2$. Therefore, for each selected vertex, $1 \le 2x_v$. Summing over the selected vertices gives:

$$|C| \le 2 \Sigma_{v \in V} x_v = 2LP^*$$

The LP relaxation allows more solutions than the original integer program, so $LP^* \le |C^*|$, where C* is an optimal integral vertex cover. Hence:

$$|C| \le 2LP^* \le 2|C^*|$$

Therefore, LP-rounding is a 2-approximation algorithm for Vertex Cover.

LP-Rounding: Worked Examples

Example 1: A Triangle

Consider a triangle with vertices A, B, and C and edges (A,B), (B,C), and (A,C). Suppose an optimal LP solution is:

$$\mathbf{xA = 0.5,\quad xB = 0.5,\quad xC = 0.5}$$

All three values are at least 1/2, so rounding selects C = {A, B, C}. This is a valid vertex cover because every edge has both endpoints selected. Its size is 3, while an optimal vertex cover of a triangle contains 2 vertices. Thus, the rounded solution satisfies the 2-approximation bound.

Example 2: A Different Fractional Solution

Suppose the LP solution is:

$$\mathbf{xA = 1,\quad xB = 0,\quad xC = 0.5}$$

Rounding selects A and C, giving C = {A, C}. Every edge of the triangle has at least one endpoint in this set, so it is a valid vertex cover. Its size is 2, which is optimal for the triangle.

Why the Approximation Bound Holds

For any edge (u, v), the constraint $x_u + x_v \geq 1$ means that both endpoints cannot have values below 1/2. Therefore, at least one endpoint is selected. Also, every selected vertex has an LP value of at least 1/2, so rounding increases the objective by at most a factor of 2 compared with the LP value.

$$\mathbf{|C| \leq 2LP^* \leq 2|C^*|}$$

This proves that the number of vertices selected by the LP-rounding algorithm is at most twice the size of an optimal vertex cover.

Comparison and Conclusion

Comparison of the Two 2-Approximation Algorithms

Feature	Edge-selection algorithm	LP-rounding algorithm
Main idea	Select an edge and add both endpoints.	Solve the relaxed LP and round values ≥ 1/2.
Intermediate result	Pairwise-disjoint selected edges (matching).	Fractional values for vertices.
Why it is a cover	All incident edges are removed after endpoints are added.	Each edge constraint forces an endpoint value ≥ 1/2.
Approximation bound	$\|C\| = 2m$ and $m \le \|C^*\|$.	$\|C\| \le 2LP^*$ and $LP^* \le \|C^*\|$.
Guarantee	$\|C\| \le 2\|C^*\|$.	$\|C\| \le 2\|C^*\|$.

Conclusion

Approximation algorithms efficiently find near-optimal solutions for difficult optimization problems and provide a mathematical guarantee on solution quality. For minimization, a k-approximation returns a cost at most k times optimal; for maximization, it returns a value at least 1/k of optimal. The Vertex Cover Problem has a simple 2-approximation algorithm based on selecting disjoint edges, and an LP-rounding algorithm that selects vertices whose fractional values are at least 1/2. Both algorithms produce a valid vertex cover C satisfying $|C| \le 2|C^*|$.

Key formulas to remember:

Minimization: $A(I) \le kA^*(I)$

Maximization: $A(I) \ge A^*(I)/k$

Vertex Cover: $|C| \le 2|C^*|$